Can ExaModels Power JuMP on GPUs?

Blessing and curse of aggressive typing

Sungho Shin
shin.mit.edu

Massachusetts Institute of Technology

JuMP-dev 2026 | Edinburgh


S. Shin
@sshin23


M. Schanen
@michel2323


A. Montoison
@amontoison


B. Legat
@blegat


M. Klamkin
@klamike


F. Pacaud
@frapac


A. Rosemberg
@andrewrosemberg


F. Holtorf
@FHoltorf


J.-B. Caillau
@jbcaillau


Archim J
@hfytr

What is ExaModels?

- Algebraic modeling system in Julia, JuMP-inspired syntax.

Example: Goddard rocket — maximize final altitude.

```
using ExaModels, NLPModelsIpopt

nh, h_0, v_0, m_0, g_0 = 200, 1.0, 0.0, 1.0, 1.0
T_c, h_c, v_c, m_c = 3.5, 500.0, 620.0, 0.6
c_e = 0.5*sqrt(g_0*h_0); m_f = m_c*m_0
D_c = 0.5*v_c*(m_0/g_0); T_max = T_c*m_0*g_0

core = ExaCore(minimize=false, concrete=Val(true))

@add_var(core, h, 0:nh; start = 1.0, lvar = 1.0)
@add_var(core, v, 0:nh; start = (i/nh*(1-i/nh) for i=0:nh))
@add_var(core, m, 0:nh; start = ((m_f-m_0)*i/nh+m_0 for i=0:nh),
         lvar = m_f, uvar = m_0)
@add_var(core, T, 0:nh; start = T_max/2, lvar = 0.0, uvar = T_max)
@add_var(core, step, 1; start = 1/nh, lvar = 0.0)

@add_obj(core, h[nh])

@add_con(core, c1, -h[i] + h[i-1]
         + 0.5*step[1]*(v[i]+v[i-1]) for i=1:nh)
@add_con(core, c2,
    -v[i] + v[i-1] + 0.5*step[1]*(
        (T[i] - D_c*v[i]^2*exp(-h_c*(h[i]-h_0))/h_0
         - m[i]*g_0*(h_0/h[i])^2)/m[i]
      + (T[i-1] - D_c*v[i-1]^2*exp(-h_c*(h[i-1]-h_0))/h_0
           - m[i-1]*g_0*(h_0/h[i-1])^2)/m[i-1])
    for i=1:nh)
@add_con(core, c3, -m[i] + m[i-1]
         - 0.5*step[1]*(T[i]+T[i-1])/c_e for i=1:nh)
@add_con(core, h[0] - h_0); @add_con(core, v[0] - v_0)
@add_con(core, m[0] - m_0); @add_con(core, m[nh] - m_f)

model = ExaModel(core)
result = ipopt(model)
```

What is ExaModels?

- Algebraic modeling system in Julia, JuMP-inspired syntax.
- Builds on `NLPModels.jl` — compatible with: `Ipopt`, `MadNLP`, `KNITRO`, `Uno`, ... (essentially all JSO-style solvers)

Example: Goddard rocket — maximize final altitude.

```
using ExaModels, NLPModelsIpopt

nh, h_0, v_0, m_0, g_0 = 200, 1.0, 0.0, 1.0, 1.0
T_c, h_c, v_c, m_c = 3.5, 500.0, 620.0, 0.6
c_e = 0.5*sqrt(g_0*h_0); m_f = m_c*m_0
D_c = 0.5*v_c*(m_0/g_0); T_max = T_c*m_0*g_0

core = ExaCore(minimize=false, concrete=Val(true))

@add_var(core, h, 0:nh; start = 1.0, lvar = 1.0)
@add_var(core, v, 0:nh; start = (i/nh*(1-i/nh) for i=0:nh))
@add_var(core, m, 0:nh; start = ((m_f-m_0)*i/nh+m_0 for i=0:nh),
         lvar = m_f, uvar = m_0)
@add_var(core, T, 0:nh; start = T_max/2, lvar = 0.0, uvar = T_max)
@add_var(core, step, 1; start = 1/nh, lvar = 0.0)

@add_obj(core, h[nh])

@add_con(core, c1, -h[i] + h[i-1]
         + 0.5*step[1]*(v[i]+v[i-1]) for i=1:nh)
@add_con(core, c2,
    -v[i] + v[i-1] + 0.5*step[1]*(
        (T[i] - D_c*v[i]^2*exp(-h_c*(h[i]-h_0))/h_0
         - m[i]*g_0*(h_0/h[i])^2)/m[i]
      + (T[i-1] - D_c*v[i-1]^2*exp(-h_c*(h[i-1]-h_0))/h_0
           - m[i-1]*g_0*(h_0/h[i-1])^2)/m[i-1])
    for i=1:nh)
@add_con(core, c3, -m[i] + m[i-1]
         - 0.5*step[1]*(T[i]+T[i-1])/c_e for i=1:nh)
@add_con(core, h[0] - h_0); @add_con(core, v[0] - v_0)
@add_con(core, m[0] - m_0); @add_con(core, m[nh] - m_f)

model = ExaModel(core)
result = ipopt(model)
```

What is ExaModels?

- Algebraic modeling system in Julia, JuMP-inspired syntax.
- Builds on `NLPModels.jl` — compatible with: `Ipopt`, `MadNLP`, `KNITRO`, `Uno`, ... (essentially all JSO-style solvers)
- Ships with its own AD system — $f(x)$, $g(x)$, $\nabla_x f(x)$, $\nabla_x g(x)$, $\nabla^2_{xx} L(x, \lambda)$

Example: Goddard rocket — maximize final altitude.

```
using ExaModels, NLPModelsIpopt

nh, h_0, v_0, m_0, g_0 = 200, 1.0, 0.0, 1.0, 1.0
T_c, h_c, v_c, m_c = 3.5, 500.0, 620.0, 0.6
c_e = 0.5*sqrt(g_0*h_0); m_f = m_c*m_0
D_c = 0.5*v_c*(m_0/g_0); T_max = T_c*m_0*g_0

core = ExaCore(minimize=false, concrete=Val(true))

@add_var(core, h, 0:nh; start = 1.0, lvar = 1.0)
@add_var(core, v, 0:nh; start = (i/nh*(1-i/nh) for i=0:nh))
@add_var(core, m, 0:nh; start = ((m_f-m_0)*i/nh+m_0 for i=0:nh),
         lvar = m_f, uvar = m_0)
@add_var(core, T, 0:nh; start = T_max/2, lvar = 0.0, uvar = T_max)
@add_var(core, step, 1; start = 1/nh, lvar = 0.0)

@add_obj(core, h[nh])

@add_con(core, c1, -h[i] + h[i-1]
         + 0.5*step[1]*(v[i]+v[i-1]) for i=1:nh)
@add_con(core, c2,
    -v[i] + v[i-1] + 0.5*step[1]*(
        (T[i] - D_c*v[i]^2*exp(-h_c*(h[i]-h_0))/h_0
         - m[i]*g_0*(h_0/h[i])^2)/m[i]
      + (T[i-1] - D_c*v[i-1]^2*exp(-h_c*(h[i-1]-h_0))/h_0
           - m[i-1]*g_0*(h_0/h[i-1])^2)/m[i-1])
    for i=1:nh)
@add_con(core, c3, -m[i] + m[i-1]
         - 0.5*step[1]*(T[i]+T[i-1])/c_e for i=1:nh)
@add_con(core, h[0] - h_0); @add_con(core, v[0] - v_0)
@add_con(core, m[0] - m_0); @add_con(core, m[nh] - m_f)

model = ExaModel(core)
result = ipopt(model)
```

What is ExaModels?

- Algebraic modeling system in Julia, JuMP-inspired syntax.
- Builds on `NLPModels.jl` — compatible with: `Ipopt`, `MadNLP`, `KNITRO`, `Uno`, ... (essentially all JSO-style solvers)
- Ships with its own AD system — $f(x)$, $g(x)$, $\nabla_x f(x)$, $\nabla_x g(x)$, $\nabla^2_{xx} L(x, \lambda)$
- **Distinguishing feature**: GPU-compatible modeling — more on this later.

Example: Goddard rocket — maximize final altitude.

```
using ExaModels, NLPModelsIpopt

nh, h_0, v_0, m_0, g_0 = 200, 1.0, 0.0, 1.0, 1.0
T_c, h_c, v_c, m_c = 3.5, 500.0, 620.0, 0.6
c_e = 0.5*sqrt(g_0*h_0); m_f = m_c*m_0
D_c = 0.5*v_c*(m_0/g_0); T_max = T_c*m_0*g_0

core = ExaCore(minimize=false, concrete=Val(true))

@add_var(core, h, 0:nh; start = 1.0, lvar = 1.0)
@add_var(core, v, 0:nh; start = (i/nh*(1-i/nh) for i=0:nh))
@add_var(core, m, 0:nh; start = ((m_f-m_0)*i/nh+m_0 for i=0:nh),
         lvar = m_f, uvar = m_0)
@add_var(core, T, 0:nh; start = T_max/2, lvar = 0.0, uvar = T_max)
@add_var(core, step, 1; start = 1/nh, lvar = 0.0)

@add_obj(core, h[nh])

@add_con(core, c1, -h[i] + h[i-1]
         + 0.5*step[1]*(v[i]+v[i-1]) for i=1:nh)
@add_con(core, c2,
    -v[i] + v[i-1] + 0.5*step[1]*(
        (T[i] - D_c*v[i]^2*exp(-h_c*(h[i]-h_0))/h_0
         - m[i]*g_0*(h_0/h[i])^2)/m[i]
      + (T[i-1] - D_c*v[i-1]^2*exp(-h_c*(h[i-1]-h_0))/h_0
           - m[i-1]*g_0*(h_0/h[i-1])^2)/m[i-1])
    for i=1:nh)
@add_con(core, c3, -m[i] + m[i-1]
         - 0.5*step[1]*(T[i]+T[i-1])/c_e for i=1:nh)
@add_con(core, h[0] - h_0); @add_con(core, v[0] - v_0)
@add_con(core, m[0] - m_0); @add_con(core, m[nh] - m_f)

model = ExaModel(core)
result = ipopt(model)
```

What is ExaModels?

- Algebraic modeling system in Julia, JuMP-inspired syntax.
- Builds on `NLPModels.jl` — compatible with: `Ipopt`, `MadNLP`, `KNITRO`, `Uno`, ... (essentially all JSO-style solvers)
- Ships with its own AD system — $f(x)$, $g(x)$, $\nabla_x f(x)$, $\nabla_x g(x)$, $\nabla^2_{xx} L(x, \lambda)$
- **Distinguishing feature**: GPU-compatible modeling — more on this later.
- **Design philosophy**: not optimized for ease-of-use, optimized for runtime performance. Aim: the fastest modeling system for building and running optimization solves.

Example: Goddard rocket — maximize final altitude.

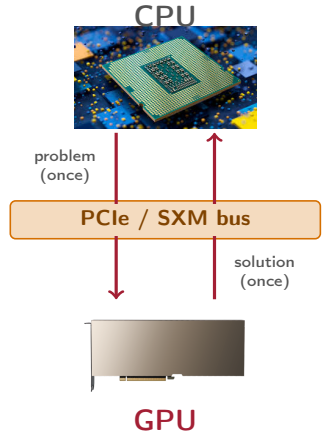
```
using ExaModels, NLPModelsIpopt

nh, h_0, v_0, m_0, g_0 = 200, 1.0, 0.0, 1.0, 1.0
T_c, h_c, v_c, m_c = 3.5, 500.0, 620.0, 0.6
c_e = 0.5*sqrt(g_0*h_0); m_f = m_c*m_0
D_c = 0.5*v_c*(m_0/g_0); T_max = T_c*m_0*g_0

core = ExaCore(minimize=false, concrete=Val(true))

@add_var(core, h, 0:nh; start = 1.0, lvar = 1.0)
@add_var(core, v, 0:nh; start = (i/nh*(1-i/nh) for i=0:nh))
@add_var(core, m, 0:nh; start = ((m_f-m_0)*i/nh+m_0 for i=0:nh),
         lvar = m_f, uvar = m_0)
@add_var(core, T, 0:nh; start = T_max/2, lvar = 0.0, uvar = T_max)
@add_var(core, step, 1; start = 1/nh, lvar = 0.0)

@add_obj(core, h[nh])

@add_con(core, c1, -h[i] + h[i-1]
         + 0.5*step[1]*(v[i]+v[i-1]) for i=1:nh)
@add_con(core, c2,
    -v[i] + v[i-1] + 0.5*step[1]*(
        (T[i] - D_c*v[i]^2*exp(-h_c*(h[i]-h_0))/h_0
         - m[i]*g_0*(h_0/h[i])^2)/m[i]
      + (T[i-1] - D_c*v[i-1]^2*exp(-h_c*(h[i-1]-h_0))/h_0
           - m[i-1]*g_0*(h_0/h[i-1])^2)/m[i-1])
    for i=1:nh)
@add_con(core, c3, -m[i] + m[i-1]
         - 0.5*step[1]*(T[i]+T[i-1])/c_e for i=1:nh)
@add_con(core, h[0] - h_0); @add_con(core, v[0] - v_0)
@add_con(core, m[0] - m_0); @add_con(core, m[nh] - m_f)

model = ExaModel(core)
result = ipopt(model)
```

Why a GPU-compatible modeling language?

LP / QP / DCP — not strictly needed

- Canonical forms $(A, b, c, \ldots) \Rightarrow$ one-shot CPU→GPU copy of problem data
- Transfers can be an overhead, but the cost is amortized over many solver iterations
- Existing GPU-native solvers in this class already work this way: `CuClarabel`, `cuPDLP+`

  $\Rightarrow$ For LP / QP / DCP, a GPU-native modeling language may not add much value

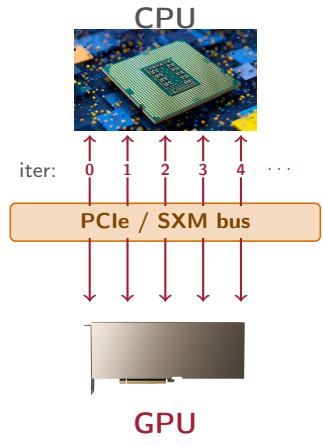


Lu et al. (2025). *cuPDLP+: A Further Enhanced GPU-Based First-Order Solver for Linear Programming.* arXiv: 2507.14051 [math]

Why a GPU-compatible modeling language?

NLP — the picture is different

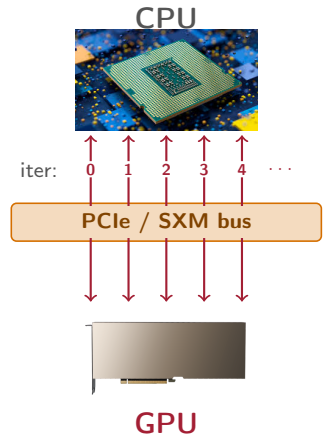
- Solver queries the modeling layer at every iteration via callbacks — $f(x)$, $g(x)$, $\nabla_x f(x)$, $\nabla_x g(x)$, $\nabla^2_{xx} L(x, \lambda)$
- Each query depends on the current iterate — no one-shot transfer
- Without GPU-native callbacks, every iteration pays a CPU↔GPU round-trip: significant per-iteration overhead, no amortization



Why a GPU-compatible modeling language?

NLP — the picture is different

- Solver queries the modeling layer at every iteration via callbacks — $f(x)$, $g(x)$, $\nabla_x f(x)$, $\nabla_x g(x)$, $\nabla^2_{xx} L(x, \lambda)$
- Each query depends on the current iterate — no one-shot transfer
- Without GPU-native callbacks, every iteration pays a CPU↔GPU round-trip: significant per-iteration overhead, no amortization
- JuMP today does not emit GPU-native callbacks. Fine: modeling infrastructure stays on CPU, as long as the callbacks themselves run on the device



The ExaModels JuMP Interface: Status Quo

```julia
using JuMP, ExaModels, CUDA, MadNLP, MadNLPGPU

# Build the JuMP model
N = 100
jm = Model()
@variable(jm, x[i=1:N], start = isodd(i) ? -1.2 : 1.0)
@objective(jm, Min, sum(100*(x[i+1] - x[i]^2)^2 + (1 - x[i])^2 for i = 1:N-1))

# Path 1 --- Direct conversion to an ExaModel
em = ExaModel(jm; backend = CUDABackend())
result = madnlp(em)

# Path 2 --- ExaModels as a JuMP optimizer
set_optimizer(jm, () -> ExaModels.Optimizer(MadNLP.madnlp, CUDABackend()))
optimize!(jm)
```

- Currently an experimental feature: it works, but with big caveats — discussed in the next slides

JuMP → ExaModels layer is slow

JuMP route

```
function build_via_jump(N)
    jm = Model()
    @variable(jm, x[i=1:N],
        start = isodd(i) ? -1.2 : 1.0)
    @objective(jm, Min,
        sum(100*(x[i+1] - x[i]^2)^2
            + (1 - x[i])^2 for i = 1:N-1))
    return ExaModel(jm)
end
```

```
julia> @btime build_via_jump(10^5);
  2.230 s (28307923 allocations: 1.06 GiB)
```

Native ExaModels

```
function build_native(N)
    core = ExaCore(concrete = Val(true))
    @add_var(core, x, N;
    start = (isodd(i) ? -1.2 : 1.0 for i = 1:N))
    @add_obj(core,
        100*(x[i+1] - x[i]^2)^2
        + (1 - x[i])^2 for i = 1:(N-1))
    return ExaModel(core)
end
```

```
julia> @btime build_native(10^5);
  309.411 us (78 allocations: 3.13 MiB)
```

- ExaModels itself is faster than JuMP, but most of the overhead is in the JuMP → ExaModels conversion layer (next slide)

JuMP → ExaModels layer is slow

JuMP route

```
function build_via_jump(N)
    jm = Model()
    @variable(jm, x[i=1:N],
        start = isodd(i) ? -1.2 : 1.0)
    @objective(jm, Min,
        sum(100*(x[i+1] - x[i]^2)^2
            + (1 - x[i])^2 for i = 1:N-1))
    return ExaModel(jm)
end
```

```
julia> @btime build_via_jump(10^5);
  2.230 s (28307923 allocations: 1.06 GiB)
```

Native ExaModels

```
function build_native(N)
    core = ExaCore(concrete = Val(true))
    @add_var(core, x, N;
    start = (isodd(i) ? -1.2 : 1.0 for i = 1:N))
    @add_obj(core,
        100*(x[i+1] - x[i]^2)^2
        + (1 - x[i])^2 for i = 1:(N-1))
    return ExaModel(core)
end
```

```
julia> @btime build_native(10^5);
  309.411 us (78 allocations: 3.13 MiB)
```

- ExaModels itself is faster than JuMP, but most of the overhead is in the JuMP → ExaModels conversion layer (next slide)
- ∼7 200× faster build, ∼350× less memory, ∼360,000× fewer allocations

JuMP → ExaModels layer is slow

JuMP route

```
function build_via_jump(N)
    jm = Model()
    @variable(jm, x[i=1:N],
        start = isodd(i) ? -1.2 : 1.0)
    @objective(jm, Min,
        sum(100*(x[i+1] - x[i]^2)^2
            + (1 - x[i])^2 for i = 1:N-1))
    return ExaModel(jm)
end
```

```
julia> @btime build_via_jump(10^5);
  2.230 s (28307923 allocations: 1.06 GiB)
```

Native ExaModels

```
function build_native(N)
    core = ExaCore(concrete = Val(true))
    @add_var(core, x, N;
    start = (isodd(i) ? -1.2 : 1.0 for i = 1:N))
    @add_obj(core,
        100*(x[i+1] - x[i]^2)^2
        + (1 - x[i])^2 for i = 1:(N-1))
    return ExaModel(core)
end
```

```
julia> @btime build_native(10^5);
  309.411 us (78 allocations: 3.13 MiB)
```

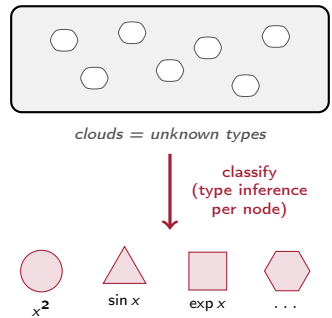
- ExaModels itself is faster than JuMP, but most of the overhead is in the JuMP → ExaModels conversion layer (next slide)
- $\sim 7\,200\times$ faster build, $\sim 350\times$ less memory, $\sim 360{,}000\times$ fewer allocations
- The gap grows with N

Where the gap comes from: JuMP unrolls into a type-erased array

```
N = 100
jm = Model()
@variable(jm, x[i=1:N],
    start = isodd(i) ? -1.2 : 1.0)
@objective(jm, Min, sum(
    100*(x[i+1] - x[i]^2)^2 + (1 - x[i])^2
    for i = 1:N-1))
```

```
julia> moi = backend(jm);

julia> obj = moi.model_cache.model.objective;

julia> obj.scalar_nonlinear.args |> typeof
Vector{Any}
```

obj.scalar_nonlinear.args :: Vector{Any}

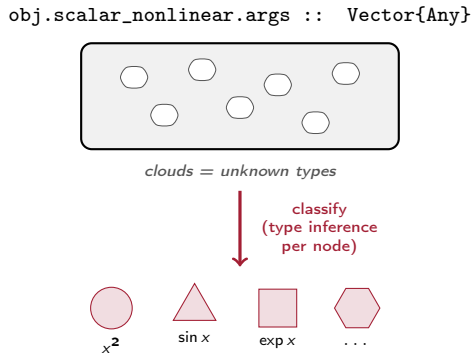


- JuMP stores the objective as an enumerated array of expression trees in a flat `Vector{Any}` — element types erased

Where the gap comes from: JuMP unrolls into a type-erased array

```
N = 100
jm = Model()
@variable(jm, x[i=1:N],
    start = isodd(i) ? -1.2 : 1.0)
@objective(jm, Min, sum(
    100*(x[i+1] - x[i]^2)^2 + (1 - x[i])^2
    for i = 1:N-1))
```

```
julia> moi = backend(jm);

julia> obj = moi.model_cache.model.objective;

julia> obj.scalar_nonlinear.args |> typeof
Vector{Any}
```

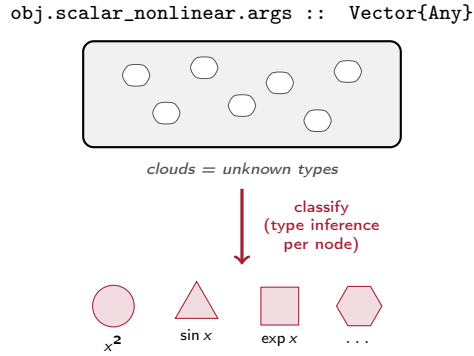


- JuMP stores the objective as an enumerated array of expression trees in a flat `Vector{Any}` — element types erased
- The ExaModels–JuMP extension walks every node, infers its type, classifies it into an operator pattern (x^2, $\sin x$, $\exp x$, $\ldots$) — per-node type inference dominates the cost

Where the gap comes from: JuMP unrolls into a type-erased array

```
N = 100
jm = Model()
@variable(jm, x[i=1:N],
    start = isodd(i) ? -1.2 : 1.0)
@objective(jm, Min, sum(
    100*(x[i+1] - x[i]^2)^2 + (1 - x[i])^2
    for i = 1:N-1))
```

```
julia> moi = backend(jm);

julia> obj = moi.model_cache.model.objective;

julia> obj.scalar_nonlinear.args |> typeof
Vector{Any}
```



- JuMP stores the objective as an enumerated array of expression trees in a flat `Vector{Any}` — element types erased
- The ExaModels–JuMP extension walks every node, infers its type, classifies it into an operator pattern (x^2, $\sin x$, $\exp x$, . . .) — per-node type inference dominates the cost
- Worse: covering all of JuMP's syntax (every operator, container, macro) is provably

One remedy: piggyback on JuMP (`GenOpt.jl` by `@blegat`)

@blegat

```julia
using GenOpt, JuMP, CUDA, MadNLP

model = Model()
@constraint(model, [i in 1:n], x[1,i] == x0[i], container = ParametrizedArray) # grouped constraint
@objective(model, Min, lazy_sum(0.5 * R[j] * u[i,j]^2 for i in 1:N, j in 1:p)) # grouped sum

set_optimizer(model, () -> GenOpt.ExaOptimizer(madnlp, CUDABackend()))
optimize!(model)
```

- Uses JuMP's own extension hooks so the algebraic patterns never get unrolled — ExaModels sees the structured representation directly

Legat (2025). *GenOpt.jl: JuMP Extension for Constraint and Term Generators.*

One remedy: piggyback on JuMP (GenOpt.jl by @blegat)

@blegat

```julia
using GenOpt, JuMP, CUDA, MadNLP

model = Model()
@constraint(model, [i in 1:n], x[1,i] == x0[i], container = ParametrizedArray) # grouped constraint
@objective(model, Min, lazy_sum(0.5 * R[j] * u[i,j]^2 for i in 1:N, j in 1:p)) # grouped sum

set_optimizer(model, () -> GenOpt.ExaOptimizer(madnlp, CUDABackend()))
optimize!(model)
```

- Uses JuMP's own extension hooks so the algebraic patterns never get unrolled — ExaModels sees the structured representation directly
- Minimal model rewrite: `container = ParametrizedArray` on `@constraint`, `lazy_sum` on `@objective`

Legat (2025). *GenOpt.jl: JuMP Extension for Constraint and Term Generators.*

One remedy: piggyback on JuMP (`GenOpt.jl` by `@blegat`)

@blegat

```julia
using GenOpt, JuMP, CUDA, MadNLP

model = Model()
@constraint(model, [i in 1:n], x[1,i] == x0[i], container = ParametrizedArray) # grouped constraint
@objective(model, Min, lazy_sum(0.5 * R[j] * u[i,j]^2 for i in 1:N, j in 1:p)) # grouped sum

set_optimizer(model, () -> GenOpt.ExaOptimizer(madnlp, CUDABackend()))
optimize!(model)
```

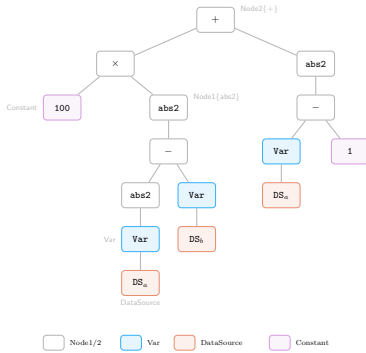
- Uses JuMP's own extension hooks so the algebraic patterns never get unrolled — ExaModels sees the structured representation directly
- Minimal model rewrite: `container = ParametrizedArray` on `@constraint`, `lazy_sum` on `@objective`
- A really cool proof-of-concept route — I'm thinking about something else next.

Legat (2025). *GenOpt.jl: JuMP Extension for Constraint and Term Generators.*

ExaModels: the type *is* the tree

```
core = ExaCore(concrete = Val(true))
@add_var(core, x, N)
@add_obj(core,
    100*(x[i-1]^2 - x[i])^2 + (x[i-1] - 1)^2
    for i = 2:N)
em = ExaModel(core)
```

```
julia> ExaModels.fulltype_display!(true);

julia> typeof(em.objs[1].f.f)
Node2{typeof(+),Node2{typeof(*),Int64,Node1{typeof(abs2),Node2{typeof
(-),Node1{typeof(abs2),Var{Node2{typeof(+),Node2{typeof(-),DataSource,
Int64},Int64}}},Var{Node2{typeof(+),DataSource,Int64}}}}},Node1{typeof(
abs2),Node2{typeof(-),Var{Node2{typeof(+),Node2{typeof(-),DataSource,
Int64},Int64}},Int64}}}
```

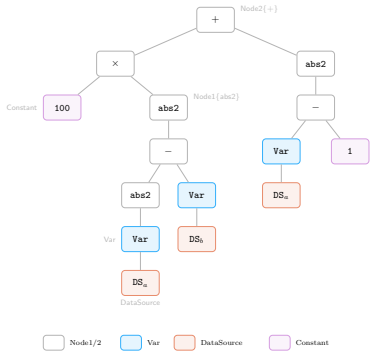


- ExaModels encodes the algebraic structure directly into the type system
- Just by looking at the type, you can read the structure of the expression

ExaModels: the type *is* the tree

```
core = ExaCore(concrete = Val(true))
@add_var(core, x, N)
@add_obj(core,
    100*(x[i-1]^2 - x[i])^2 + (x[i-1] - 1)^2
    for i = 2:N)
em = ExaModel(core)
```

```
julia> ExaModels.fulltype_display!(true);

julia> typeof(em.objs[1].f.f)
Node2{typeof(+),Node2{typeof(*),Int64,Node1{typeof(abs2),Node2{typeof
(-),Node1{typeof(abs2),Var{Node2{typeof(+),Node2{typeof(-),DataSource,
Int64},Int64}}},Var{Node2{typeof(+),DataSource,Int64}}}}},Node1{typeof(
abs2),Node2{typeof(-),Var{Node2{typeof(+),Node2{typeof(-),DataSource,
Int64},Int64}},Int64}}}
```

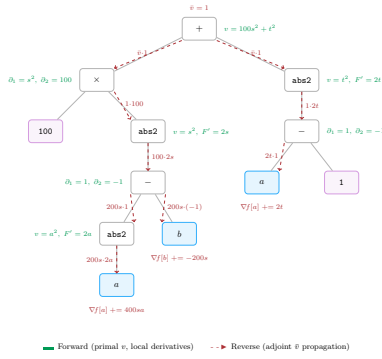


- ExaModels encodes the algebraic structure directly into the type system
- Just by looking at the type, you can read the structure of the expression
- At compile time the structure is fully known ⇒ model creation, AD, and evaluation are specialized for this algebraic expression

ExaModels: the type *is* the tree

```
core = ExaCore(concrete = Val(true))
@add_var(core, x, N)
@add_obj(core,
    100*(x[i-1]^2 - x[i])^2 + (x[i-1] - 1)^2
    for i = 2:N)
em = ExaModel(core)
```

```
julia> ExaModels.fulltype_display!(true);

julia> typeof(em.objs[1].f.f)
Node2{typeof(+),Node2{typeof(*),Int64,Node1{typeof(abs2),Node2{typeof
(-),Node1{typeof(abs2),Var{Node2{typeof(+),Node2{typeof(-),DataSource,
Int64},Int64}}},Var{Node2{typeof(+),DataSource,Int64}}}}},Node1{typeof(
abs2),Node2{typeof(-),Var{Node2{typeof(+),Node2{typeof(-),DataSource,
Int64},Int64}},Int64}}}
```

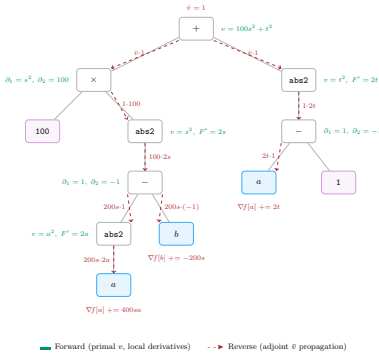


- Forward (green) and reverse (red) pass propagate through the *same typed tree*

ExaModels: the type *is* the tree

```
core = ExaCore(concrete = Val(true))
@add_var(core, x, N)
@add_obj(core,
    100*(x[i-1]^2 - x[i])^2 + (x[i-1] - 1)^2
    for i = 2:N)
em = ExaModel(core)
```

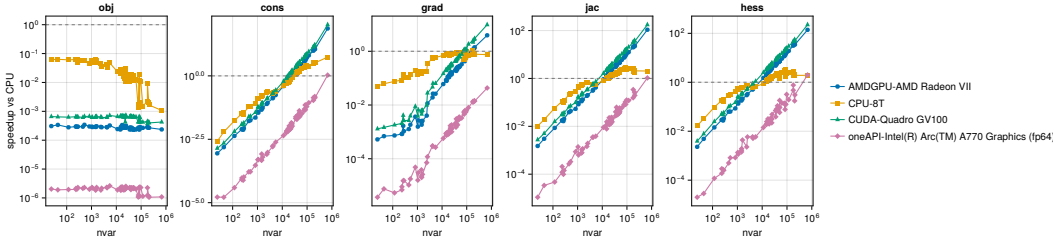
```
julia> ExaModels.fulltype_display!(true);

julia> typeof(em.objs[1].f.f)
Node2{typeof(+),Node2{typeof(*),Int64,Node1{typeof(abs2),Node2{typeof
(-),Node1{typeof(abs2),Var{Node2{typeof(+),Node2{typeof(-),DataSource,
Int64},Int64}}},Var{Node2{typeof(+),DataSource,Int64}}}}},Node1{typeof(
abs2),Node2{typeof(-),Var{Node2{typeof(+),Node2{typeof(-),DataSource,
Int64},Int64}},Int64}}}
```



- Forward (green) and reverse (red) pass propagate through the *same typed tree*
- Both passes can be compiled for this `Node2{...}` type
  $\Rightarrow$ at runtime, no type inference on the tree

Blessing: Portability and performance

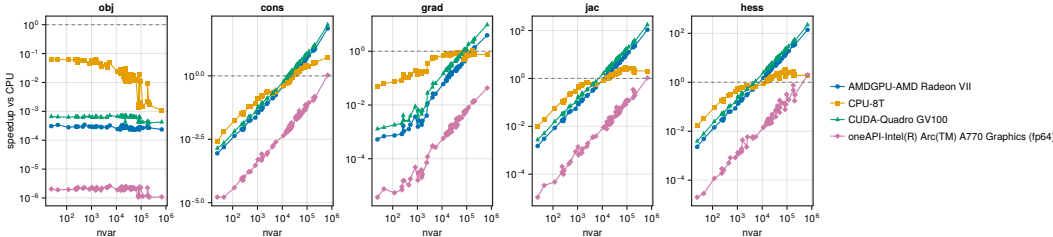
AD-callback speedup vs CPU-1T baseline, AC OPF (PGLIB-OPF)



- Tree complexity decoupled from the array level: only data-level parallelism is applied; per-data-point ops are scalar Julia, fully type-inferable

Blessing: Portability and performance

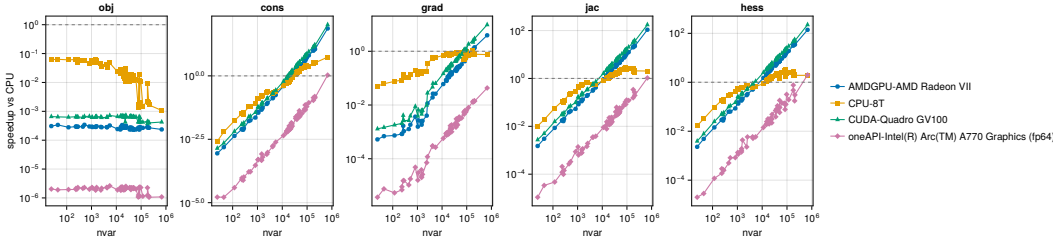
AD-callback speedup vs CPU-1T baseline, AC OPF (PGLIB-OPF)



- Tree complexity decoupled from the array level: only data-level parallelism is applied; per-data-point ops are scalar Julia, fully type-inferable
- Same model code runs on `CPU()`, `CUDABackend()`, `ROCBackend()`, `oneAPIBackend()`, `MetalBackend()` — portable across GPU architectures via the JuliaGPU stack: `KernelAbstractions.jl` over `GPUCompiler.jl`

Blessing: Portability and performance

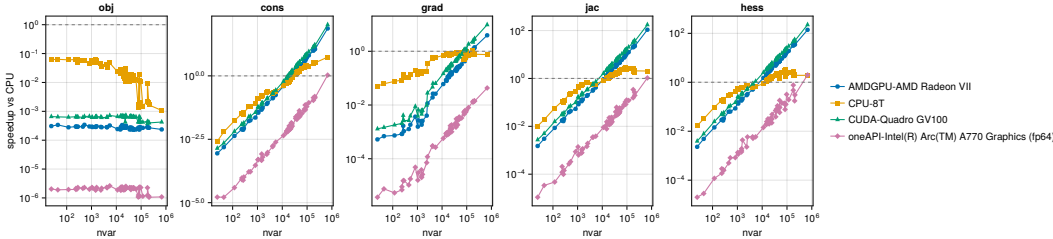
AD-callback speedup vs CPU-1T baseline, AC OPF (PGLIB-OPF)



- Tree complexity decoupled from the array level: only data-level parallelism is applied; per-data-point ops are scalar Julia, fully type-inferable
- Same model code runs on `CPU()`, `CUDABackend()`, `ROCBackend()`, `oneAPIBackend()`, `MetalBackend()` — portable across GPU architectures via the JuliaGPU stack: `KernelAbstractions.jl` over `GPUCompiler.jl`
- Jacobian / Hessian: CUDA, AMDGPU reach $> 100\times$ at $n_{\text{var}} \sim 10^6$; `oneAPI` fp64 on Arc A770 is emulated $\Rightarrow$ hardware limit, not a software gap

Blessing: Portability and performance

AD-callback speedup vs CPU-1T baseline, AC OPF (PGLIB-OPF)



- Tree complexity decoupled from the array level: only data-level parallelism is applied; per-data-point ops are scalar Julia, fully type-inferable
- Same model code runs on `CPU()`, `CUDABackend()`, `ROCBackend()`, `oneAPIBackend()`, `MetalBackend()` — portable across GPU architectures via the JuliaGPU stack: `KernelAbstractions.jl` over `GPUCompiler.jl`
- Jacobian / Hessian: CUDA, AMDGPU reach $> 100\times$ at $n_{\text{var}} \sim 10^6$; `oneAPI` fp64 on Arc A770 is emulated $\Rightarrow$ hardware limit, not a software gap
- More benchmark results presented at SIAM Optimization 2026 (MS369, Fri June 5)

Blessing: AOT compile to a standalone binary

```
module MySolve
using ExaModels, NLPModelsIpoptLite
function (@main)(ARGS)
    N = 100
    core = ExaCore(concrete = Val(true))
    @add_var(core, x, N; start = (isodd(i) ? -1.2 : 1.0 for i = 1:N))
    @add_obj(core, 100*(x[i+1] - x[i]^2)^2 + (1 - x[i])^2 for i = 1:N-1)
    result = ipopt(ExaModel(core); print_level = 0)
    println(Core.stdout, "status = ", result.status)
    return result.status == 0 ? 0 : 1
end
end # module
```

```
$ juliac --trim=safe -o rosenbrock MySolve.jl   # compile to a standalone executable
$ ./rosenbrock
status = first_order
```

- Aggressive typing applies at the whole `ExaModel`, not just the expression tree

Blessing: AOT compile to a standalone binary

```
module MySolve
using ExaModels, NLPModelsIpoptLite
function (@main)(ARGS)
    N = 100
    core = ExaCore(concrete = Val(true))
    @add_var(core, x, N; start = (isodd(i) ? -1.2 : 1.0 for i = 1:N))
    @add_obj(core, 100*(x[i+1] - x[i]^2)^2 + (1 - x[i])^2 for i = 1:N-1)
    result = ipopt(ExaModel(core); print_level = 0)
    println(Core.stdout, "status = ", result.status)
    return result.status == 0 ? 0 : 1
end
end # module
```

```
$ juliac --trim=safe -o rosenbrock MySolve.jl   # compile to a standalone executable
$ ./rosenbrock
status = first_order
```

- Aggressive typing applies at the whole `ExaModel`, not just the expression tree
- Possible because the type is fully inferable at the modeling-function level

Blessing: AOT compile to a standalone binary

```
module MySolve
using ExaModels, NLPModelsIpoptLite
function (@main)(ARGS)
    N = 100
    core = ExaCore(concrete = Val(true))
    @add_var(core, x, N; start = (isodd(i) ? -1.2 : 1.0 for i = 1:N))
    @add_obj(core, 100*(x[i+1] - x[i]^2)^2 + (1 - x[i])^2 for i = 1:N-1)
    result = ipopt(ExaModel(core); print_level = 0)
    println(Core.stdout, "status = ", result.status)
    return result.status == 0 ? 0 : 1
end
end # module
```

```
$ juliac --trim=safe -o rosenbrock MySolve.jl   # compile to a standalone executable
$ ./rosenbrock
status = first_order
```

- Aggressive typing applies at the whole `ExaModel`, not just the expression tree
- Possible because the type is fully inferable at the modeling-function level
- Aggressive typing + full inlining: LLVM IR is one giant block for the typed `ExaModel`

Curse: compile-once per algebraic pattern

```julia
function build_v1(N)
    core = ExaCore(concrete = Val(true))
    @add_var(core, x, N)
    @add_obj(core,
        x[i]^2 for i = 1:N)
    return ExaModel(core)
end
```

```julia
function build_v2(N)
    core = ExaCore(concrete = Val(true))
    @add_var(core, x, N)
    @add_obj(core,
        x[i]^2 + sin(x[i]) for i = 1:N)
    return ExaModel(core)
end
```

Curse: compile-once per algebraic pattern

```julia
function build_v1(N)
    core = ExaCore(concrete = Val(true))
    @add_var(core, x, N)
    @add_obj(core,
        x[i]^2 for i = 1:N)
    return ExaModel(core)
end
```

```julia
function build_v2(N)
    core = ExaCore(concrete = Val(true))
    @add_var(core, x, N)
    @add_obj(core,
        x[i]^2 + sin(x[i]) for i = 1:N)
    return ExaModel(core)
end
```

```
julia> @time em = build_v1(1000);     # v1 first call: fresh pattern compile
  0.088402 seconds (316.07 k allocations: 15.612 MiB, 99.96% compilation time)

julia> @time em = build_v1(10000);    # different N, same pattern: cached, no recompile
  0.000048 seconds (75 allocations: 323.531 KiB)

julia> @time em = build_v2(1000);     # different pattern: fresh compile
  0.094173 seconds (354.10 k allocations: 17.438 MiB, 99.95% compilation time)
```

Curse: compile-once per algebraic pattern

```
function build_v1(N)
    core = ExaCore(concrete = Val(true))
    @add_var(core, x, N)
    @add_obj(core,
        x[i]^2 for i = 1:N)
    return ExaModel(core)
end
```

```
function build_v2(N)
    core = ExaCore(concrete = Val(true))
    @add_var(core, x, N)
    @add_obj(core,
        x[i]^2 + sin(x[i]) for i = 1:N)
    return ExaModel(core)
end
```

```
julia> @time em = build_v1(1000);     # v1 first call: fresh pattern compile
  0.088402 seconds (316.07 k allocations: 15.612 MiB, 99.96% compilation time)

julia> @time em = build_v1(10000);    # different N, same pattern: cached, no recompile
  0.000048 seconds (75 allocations: 323.531 KiB)

julia> @time em = build_v2(1000);     # different pattern: fresh compile
  0.094173 seconds (354.10 k allocations: 17.438 MiB, 99.95% compilation time)
```

- Same algebraic pattern $\Rightarrow$ cached, instant rebuild

Curse: compile-once per algebraic pattern

```
function build_v1(N)
    core = ExaCore(concrete = Val(true))
    @add_var(core, x, N)
    @add_obj(core,
        x[i]^2 for i = 1:N)
    return ExaModel(core)
end
```

```
function build_v2(N)
    core = ExaCore(concrete = Val(true))
    @add_var(core, x, N)
    @add_obj(core,
        x[i]^2 + sin(x[i]) for i = 1:N)
    return ExaModel(core)
end
```

```
julia> @time em = build_v1(1000);     # v1 first call: fresh pattern compile
  0.088402 seconds (316.07 k allocations: 15.612 MiB, 99.96% compilation time)

julia> @time em = build_v1(10000);    # different N, same pattern: cached, no recompile
  0.000048 seconds (75 allocations: 323.531 KiB)

julia> @time em = build_v2(1000);     # different pattern: fresh compile
  0.094173 seconds (354.10 k allocations: 17.438 MiB, 99.95% compilation time)
```

- Same algebraic pattern ⇒ cached, instant rebuild
- A slight change (one extra term) ⇒ everything recompiles (∼99% compilation time)

Maybe an alternative: ExaModels as a JuMP backend for GPU acceleration

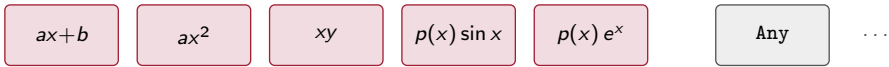
Idea: a small number of fully-typed buckets

- Don't ask JuMP to be fully type-stable. Give it a small number of buckets — each potentially large — whose contents are fully typed and concrete

Maybe an alternative: ExaModels as a JuMP backend for GPU acceleration

Idea: a small number of fully-typed buckets

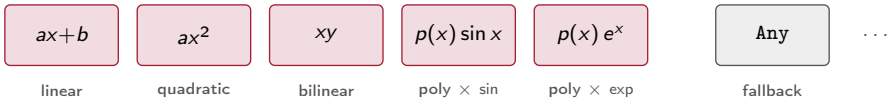
- Don't ask JuMP to be fully type-stable. Give it a small number of buckets — each potentially large — whose contents are fully typed and concrete
- ∼10 patterns of practical significance: linear, quadratic, bilinear, poly × sin, poly × exp, . . . , plus an Any fallback bucket for anything else



Maybe an alternative: ExaModels as a JuMP backend for GPU acceleration

Idea: a small number of fully-typed buckets

- Don't ask JuMP to be fully type-stable. Give it a small number of buckets — each potentially large — whose contents are fully typed and concrete
- ∼10 patterns of practical significance: linear, quadratic, bilinear, poly × sin, poly × exp, . . . , plus an Any fallback bucket for anything else

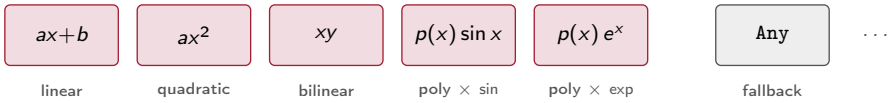


- JuMP authors against these blocks; ExaModels dispatches statically on the common patterns; the `Any` bucket keeps today's per-element inference path

Maybe an alternative: ExaModels as a JuMP backend for GPU acceleration

Idea: a small number of fully-typed buckets

- Don't ask JuMP to be fully type-stable. Give it a small number of buckets — each potentially large — whose contents are fully typed and concrete
- ∼10 patterns of practical significance: linear, quadratic, bilinear, poly × sin, poly × exp, . . . , plus an Any fallback bucket for anything else



- JuMP authors against these blocks; ExaModels dispatches statically on the common patterns; the `Any` bucket keeps today's per-element inference path
- Implication for JuMP: a move from purely interpreted model construction toward more typed + compiled — good and bad. Just an idea.

Summary

https://github.com/exanauts/ExaModels.jl

Slides: https://sshin23.github.io/slides-jump-dev-2026/main.pdf

- Encoding the algebraic structure in the type buys us GPU-native callbacks, portability, and AOT compilation
- The trade-off: all the burden goes onto the compiler — great when patterns are stable, expensive when they change

Summary

https://github.com/exanauts/ExaModels.jl

Slides: https://sshin23.github.io/slides-jump-dev-2026/main.pdf

- Encoding the algebraic structure in the type buys us GPU-native callbacks, portability, and AOT compilation
- The trade-off: all the burden goes onto the compiler — great when patterns are stable, expensive when they change

Discussion

We don't need to move all of JuMP onto the GPU. The win is GPU-native callbacks eliminating the per-iteration round-trip. A slightly more type-stable JuMP tree would make the bridge cheap.

Summary

https://github.com/exanauts/ExaModels.jl

Slides: https://sshin23.github.io/slides-jump-dev-2026/main.pdf

- Encoding the algebraic structure in the type buys us GPU-native callbacks, portability, and AOT compilation
- The trade-off: all the burden goes onto the compiler — great when patterns are stable, expensive when they change

Discussion

We don't need to move all of JuMP onto the GPU. The win is GPU-native callbacks eliminating the per-iteration round-trip. A slightly more type-stable JuMP tree would make the bridge cheap.

Continuing this thread at SIAM Optimization 2026

"The State of ExaModels" — MS369, Fri June 5, 11:10 BST.

Can ExaModels Power JuMP on GPUs?

Blessing and curse of aggressive typing

Sungho Shin
shin.mit.edu

Massachusetts Institute of Technology

JuMP-dev 2026 | Edinburgh


S. Shin
@sshin23


M. Schanen
@michel2323


A. Montoison
@amontoison


B. Legat
@blegat


M. Klamkin
@klamike


F. Pacaud
@frapac


A. Rosemberg
@andrewrosemberg


F. Holtorf
@FHoltorf


J.-B. Caillau
@jbcaillau


Archim J
@hfytr